Abstract Factory ist ein kreatives Entwurfsmuster, das das Problem der Erstellung ganzer Produktfamilien ohne Angabe ihrer konkreten Klassen löst.

Abstract Factory definiert eine Schnittstelle zur Erstellung aller unterschiedlichen Produkte, überlässt die eigentliche Produkterstellung jedoch konkreten Factory-Klassen. Jeder Factory-Typ entspricht einer bestimmten Produktvariante.

Der Clientcode ruft die Erstellungsmethoden eines Factory-Objekts auf, anstatt Produkte direkt mit einem Konstruktoraufruf (`new`Operator) zu erstellen. Da eine Factory einer einzigen Produktvariante entspricht, sind alle ihre Produkte kompatibel.

Clientcode arbeitet mit Fabriken und Produkten ausschließlich über deren abstrakte Schnittstellen. Dadurch kann der Clientcode mit allen vom Fabrikobjekt erstellten Produktvarianten arbeiten. Sie erstellen einfach eine neue konkrete Fabrikklasse und übergeben diese an den Clientcode.

Anwendungsbeispiele: Das Abstract Factory-Muster ist in Dart-Code weit verbreitet. Viele Frameworks und Bibliotheken nutzen es, um ihre Standardkomponenten zu erweitern und anzupassen.

Identifikation: Das Muster ist leicht an Methoden zu erkennen, die ein Factory-Objekt zurückgeben. Die Factory wird dann zur Erstellung spezifischer Unterkomponenten verwendet.

```dart
// Creational Design Patterns: Factory Method

// Abstraktes Produkt Ticket
abstract class Ticket {
  String getTicketType();
  String getPrice();
}

// Konkrete Implementierungen des Produkts Ticket
class ErwachseneTicket implements Ticket {
  @override
  String getTicketType() => 'Erwachsene Ticket';
  @override
  String getPrice() => '10 Euro';
}

class SchuelerTicket implements Ticket {
  @override
  String getTicketType() => 'Schüler Ticket';
  @override
  String getPrice() => '5 Euro';
}

// Abstrakte Fabrik TicketFactory
abstract class TicketFactory {
  Ticket createTicket(int age);
}

// Konkrete Fabrik für Bustickets
class BusTicketFactory implements TicketFactory {
  @override
  Ticket createTicket(int age) {
    if (age < 18) {
      return SchuelerTicket();
    } else {
      return ErwachseneTicket();
    }
  }
}

void main() {
  final age = 18;
  TicketFactory factory = BusTicketFactory();
  Ticket ticket = factory.createTicket(age);

  print('Ihr Ticket: ${ticket.getTicketType()}'); // Erwachsene Ticket
  print('Preis: ${ticket.getPrice()}'); // 10 Euro
}
```